

REPORT ATTIVITÀ PROGETTUALE

Sicurezza dell'Informazione

Progetto: Network Pentest Tool

CryptoNet Labs

A.R.E.S.

AUTOMATED RECONNAISSANCE & ENUMERATION SYSTEM

Candidato:

Stefano Casini
Università di Bologna

Tutor Aziendale:

Vincenzo Mucciante
CryptonetLabs

14 April 2026

Indice

1	Introduzione	3
1.1	Contesto del progetto	3
1.2	Obiettivi del Progetto	5
2	Analisi del Problema e Strumenti	6
2.1	Modalità di Scansione e Vincoli	6
2.2	Selezione degli Strumenti	6
2.3	Tools Recap	8
3	Architettura e Implementazione	9
3.1	Panoramica del Sistema	9
3.2	Pattern di Estensibilità	9
3.3	Pipeline di Esecuzione	11
3.4	Interfaccia Utente (CLI) e Configurazione	13
4	Configurazione e Caso d'Uso	14
4.1	Definizione dello Scenario	14
4.2	Analisi della Configurazione	14
4.3	Struttura dell'Output e Analisi dei Risultati	16
5	Conclusioni	18
5.1	Raggiungimento degli obiettivi	18
5.2	Limiti attuali e Sviluppi futuri	18

1 – Introduzione

1.1 Contesto del progetto

Nel contesto delle attività di **Network Penetration Testing**, l'analisi preliminare dell'infrastruttura rappresenta una fase fondamentale per comprendere la superficie di attacco, l'esposizione dei servizi e le potenziali vulnerabilità.

Questa fase, tecnicamente definita come **Reconnaissance**, può essere condotta secondo due paradigmi distinti ma complementari: la modalità **Passiva** e quella **Attiva**. Sebbene entrambe condividano l'obiettivo di raccogliere informazioni sul sistema target, differiscono sostanzialmente per le metodologie di acquisizione e per il livello di interazione con esso.

Nella **Passive Reconnaissance**, il pentester non interagisce direttamente con l'infrastruttura bersaglio. Le informazioni vengono invece reperite attraverso fonti pubbliche (OSINT) o servizi di scansione massiva come **Shodan** o **Censys**. Il vantaggio principale di questo approccio risiede nella totale invisibilità: il Pentester non genera traffico diretto verso la rete vittima, rendendo l'attività non tracciabile dai sistemi di difesa (IDS/IPS). Tuttavia, il rovescio della medaglia è l'affidabilità delle informazioni ottenute, che potrebbero essere obsolete, incomplete o non riflettere lo stato attuale della configurazione di rete.

Al contrario, l'**Active Reconnaissance** prevede un'interazione diretta tra la macchina dell'attaccante e l'infrastruttura target attraverso l'invio di pacchetti di rete. In questo scenario, l'utilizzo di strumenti specializzati permette di ottenere un'istantanea precisa e aggiornata dello stato dei servizi. Tuttavia, questa precisione comporta un costo in termini di rilevabilità. Esiste infatti un **trade-off** diretto tra la profondità dell'analisi e il «rumore» generato: scansioni aggressive offrono dettagli maggiori ma aumentano esponenzialmente la probabilità di essere rilevati e bloccati dai sistemi di sicurezza perimetrali.

Tuttavia, la scelta tra metodologie attive o passive non è dettata esclusivamente da preferenze tecniche, ma è subordinata a vincoli formali ben precisi. Ogni attività di pentesting è infatti disciplinata dalle **Rules of Engagement** (RoE), un documento contrattuale concordato preventivamente tra il committente (l'organizzazione proprietaria dell'infrastruttura) e l'esecutore (il **Pentester** o il **Red Team**).

Le RoE definiscono il perimetro legale e operativo dell'intervento, chiarendo esplicitamente le azioni consentite. Tale accordo è essenziale per garantire la conformità normativa e tutelare la continuità del business, prevenendo disservizi o interruzioni accidentali dei sistemi.

Queste regole influenzano drasticamente la strategia di **Reconnaissance**:

- In scenari permissivi, le RoE possono autorizzare scansioni aggressive per testare la resilienza dei sistemi.
- In contesti critici o di produzione, le RoE possono imporre finestre temporali ristrette o vietare categoricamente l'interazione attiva (scansioni), rendendo l'approccio passivo l'unica via percorribile.

Oltre ai vincoli normativi, occorre considerare la **crescente complessità** delle reti moderne, caratterizzate da architetture ibride, servizi cloud distribuiti e perimetri sempre meno definiti, rende inadeguato l'utilizzo di un approccio monolitico. Nessun singolo strumento, per quanto avanzato, è oggi in grado di coprire efficacemente l'intero **spettro di analisi richiesto**.

Per ottenere una panoramica completa dell'infrastruttura, l'analista è costretto ad adottare una strategia dove si **combinano strumenti verticali specializzati**: scanner volumetrici per la velocità, enumerator di servizi per la precisione e motori OSINT per la furtività. Tuttavia, se da un lato tale eterogeneità garantisce la copertura tecnica necessaria, dall'altro introduce notevoli inefficienze operative, dovute alla mancanza di interoperabilità tra i diversi applicativi.

Questa assenza di standard comuni costringe l'analista a un'onerosa gestione manuale, caratterizzata da problematiche quali:

- **Eterogeneità dei Formati**: ogni strumento esporta i risultati secondo standard differenti (XML per Nmap, JSON proprietario per Masscan/Smap, formati Greppable), impedendo un parsing unificato immediato.
- **Incoerenza Strutturale dei Dati**: mentre alcuni tool aggregano i risultati per host (struttura gerarchica), altri producono flussi di dati grezzi non ordinati (struttura a eventi), rendendo complessa la correlazione dei servizi esposti sullo stesso indirizzo IP.
- **Gestione Caotica degli Artifact**: l'esecuzione di comandi multipli genera una frammentazione di file difficili da catalogare e associare univocamente a specifici target, timestamp o tipologie di scansione (es. discovery vs deep scan).
- **Mancanza di una Struttura Dati Aggregata**: l'assenza di un output unificato costringe l'analista a consultare frammenti di dati sparsi, rendendo difficile l'interpretazione immediata dei risultati. Inoltre, la mancanza di uno schema standardizzato impedisce di utilizzare i dati raccolti come input diretto per ulteriori strumenti di automazione.

1.2 Obiettivi del Progetto

In risposta alle criticità appena esposte, l'attività progettuale si è posta come obiettivo primario la realizzazione di **A.R.E.S. (Automated Reconnaissance & Enumeration System)**, un framework di orchestrazione che mira a colmare il divario tra la raccolta dati manuale e l'automazione industriale.

Nello specifico, il progetto persegue i seguenti obiettivi tecnici e funzionali:

- **Orchestrazione Ibrida:** integrare in un unico flusso operativo strumenti di scansione attiva (Nmap, Masscan) e sorgenti di intelligence passiva (Smap), gestendone l'esecuzione concorrente per massimizzare le performance.
- **Normalizzazione e Fusione dei Dati:** implementare un motore di parsing in grado di tradurre output eterogenei (XML, JSON, Raw) in un modello dati unificato, applicando logiche di de-duplicazione e risoluzione dei conflitti (Data Fusion).
- **Astrazione Operativa:** mascherare la complessità dei singoli comandi sottostanti tramite un'interfaccia di configurazione dichiarativa (YAML), permettendo all'operatore di definire **cosa** scansionare senza preoccuparsi del **come**.
- **Interoperabilità:** produrre un output strutturato (JSON standardizzato) che sia immediatamente leggibile («Human-readable») ma pronto per l'ingestione in pipeline di sicurezza automatizzate (SIEM, Vulnerability Scanners).
- **Architettura Modulare ed Estensibile:** garantire la manutenibilità del codice attraverso l'uso di Design Pattern (Strategy), permettendo l'aggiunta futura di nuovi moduli di scansione senza alterare il core dell'applicazione.

Il risultato atteso è una piattaforma che riduca drasticamente i tempi di **Reconnaissance**, aumentando al contempo la precisione e la coerenza dei dati raccolti rispetto all'approccio manuale tradizionale.

2 – Analisi del Problema e Strumenti

2.1 Modalità di Scansione e Vincoli

Nel **Penetration Testing** è fondamentale operare nel rispetto delle **Rules of Engagement** (RoE), ovvero l'insieme di regole e perimetri concordati preventivamente tra il committente e l'analista. Questi accordi determinano il livello di intrusività consentito. Di conseguenza, il framework è stato concepito per garantire la flessibilità necessaria ad adattarsi a qualsiasi scenario operativo, permettendo di orchestrare scansioni sia di natura **attiva** che **passiva** in base ai permessi accordati.

Nello specifico, per quanto riguarda gli strumenti di scansione attiva, il framework è stato progettato per garantire un controllo granulare sulle modalità di esecuzione, permettendo all'operatore di bilanciare dinamicamente il **trade-off** tra profondità informativa e rilevabilità (o «rumore»).

Un altro aspetto chiave dell'architettura di A.R.E.S. è il disaccoppiamento tra il motore di orchestrazione e le strategie di scansione. Difatti le modalità di scansione non sono definite rigidamente nel codice (**hardcoded**), ma sono descritte nel file di configurazione. Ciò garantisce il pieno controllo al pentester che ha così la possibilità di modificare e/o creare nuovi profili di scansione, per ogni singolo tool, in modo semplice e rapido.

Dal punto di vista dell'ambiente di esecuzione, il progetto può operare solamente su **sistemi Linux nativi**. L'utilizzo di ambienti virtualizzati, quali **WSL (Windows Subsystem for Linux)** o **Macchine Virtuali**, ha evidenziato limitazioni critiche dovute all'astrazione del livello fisico, che impedisce l'accesso diretto alle interfacce di rete necessarie per l'iniezione di pacchetti a basso livello. Questa restrizione rende inaffidabile, o del tutto impossibile, l'iniezione asincrona di pacchetti ad alta velocità, indispensabile per il funzionamento di strumenti come Masscan.

2.2 Selezione degli Strumenti

La selezione dei tool integrati risponde all'esigenza di coprire l'intero spettro delle attività di ricognizione, integrando metodologie complementari di natura attiva e passiva.

2.2.1 Nmap: (Active / Deep Recon & Enumeration)

Lo standard industriale per l'audit di sicurezza, impiegato nel framework per l'analisi approfondita degli host precedentemente individuati.

- **Advanced Port Scanning:** determina con precisione lo stato delle porte e interroga i servizi per identificarne nome e versione specifica (Service Versioning).
- **OS Detection:** analizza le risposte dello stack TCP/IP (fingerprinting) per identificare con elevata probabilità il sistema operativo del target e la sua versione.

- **Vulnerability Detection:** Nmap offre funzionalità avanzate di analisi tramite il modulo **Nmap Scripting Engine (NSE)**. Tale componente abilita l'esecuzione di script (scritti in **Lua**) per automatizzare specifiche attività durante la scansione, quali il rilevamento di vulnerabilità note (CVE), backdoor e misconfigurazioni di servizio. Oltre alla possibilità di creare script personalizzati, il tool include nativamente una vasta libreria di controlli pronti all'uso.
- **Controllo Granulare:** permette di calibrare perfettamente l'intensità della scansione (tramite i Timing Templates -T) per bilanciare velocità e furtività.

2.2.2 Masscan: (Active / Discovery & Fast Port Scanning)

Selezionato per la sua architettura asincrona capace di trasmettere milioni di pacchetti al secondo, bypassando i limiti dello stack TCP/IP del kernel.

- **Funzionalità Chiave:** esegue scansioni TCP e UDP su vasti range (interi subnet /16 o /8) in pochi secondi.
- **Banner Grabbing:** oltre a rilevare lo stato della porta (Open/Closed), è configurato per stabilire la connessione e recuperare il banner del servizio, fornendo una prima classificazione rapida senza il peso computazionale di una scansione completa.

2.2.3 Smap: (Passive / Shodan Wrapper)

Rappresenta l'interfaccia per l'intelligence passiva. Emulando la sintassi di Nmap, agisce come un «ponte» verso le API di **Shodan**.

Shodan è una delle principali piattaforme di **Internet Intelligence** al mondo: un servizio che monitora e scansiona costantemente l'intero spazio di indirizzamento globale (IPv4), indicizzando le informazioni restituite dai dispositivi connessi (banner, servizi, versioni software). In ottica di **Passive Reconnaissance**, Shodan rappresenta la fonte autoritaria per eccellenza: interrogando i suoi database, l'operatore ottiene informazioni sull'infrastruttura target basandosi su scansioni già effettuate dalla piattaforma, evitando quindi qualsiasi interazione diretta con la rete vittima.

L'integrazione di Smap nel framework garantisce due vantaggi strategici:

- **Stealth Assoluto:** Interroga i database pubblici (FreeDB di Shodan) per recuperare porte aperte, banner storici e tag (es. «cloud», «vpn») senza inviare alcun pacchetto all'infrastruttura target.
- **Validazione Storica:** utile per confrontare lo stato attuale della rete (visto da Nmap) con quello storico indicizzato, evidenziando eventuali disallineamenti o servizi esposti in precedenza.

2.3 Tools Recap

Strumento	Tipologia	Funzionalità Chiave
Nmap	Attivo (Deep Enum.)	• Precisione: Identificazione puntuale di versioni (Service Versioning) e OS. • Scripting: Rilevamento vulnerabilità tramite modulo NSE (Lua). • Flessibilità: Ampia personalizzazione tramite flag e timing templates.
Masscan	Attivo (Volumetric)	• Velocità: Scansione asincrona ad altissimo rateo (milioni di pacchetti/sec). • Architettura: Bypass dello stack TCP/IP del kernel (Raw Sockets). • Copertura: Ideale per discovery rapida su intere subnet (/16, /8).
Smap	Passivo (OSINT)	• Stealth: Nessuna interazione diretta con il bersaglio (Zero-Touch). • Fonte Dati: Utilizzo delle API di Shodan per recuperare banner indicizzati. • Analisi: Utile per validazione storica e confronto con lo stato attuale.

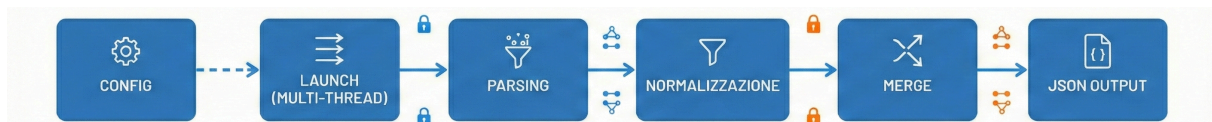
Tabella 1: Confronto delle caratteristiche operative degli strumenti analizzati

3 – Architettura e Implementazione

3.1 Panoramica del Sistema

A.R.E.S. è basato su un'architettura modulare scritta in Python. L'obiettivo primario è il disaccoppiamento tra l'esecuzione dei comandi (**Launcher**), l'interpretazione dei dati (**Parser**) e l'aggregazione finale (**Merger**).

Il flusso di esecuzione segue una pipeline sequenziale:



La struttura del progetto riflette questa logica:

- **modules/**: contiene i plugin per ogni tool (es. `nmap/`, `masscan/`).
- **utils/**: librerie trasversali per CLI, file system e merging.
- **config.yml**: centralizza parametri e chiavi API.

3.2 Pattern di Estensibilità

Per permettere l'aggiunta di nuovi tool senza riscrivere il core, il sistema utilizza due classi astratte (Interfacce) che fungono da contratto.

3.2.1 Classe **CommandGenerator**

Ogni modulo deve implementare questa classe per definire come i parametri utente vengono tradotti in comandi shell.

```
from abc import ABC, abstractmethod
from pathlib import Path
class CommandGenerator(ABC):
    def __init__(self, output_dir: Path):
        self.output_dir = output_dir
    @abstractmethod
    def generate_commands(self, config: dict) -> list:
        pass
```

- **__init__(outputpath)**: riceve il percorso dove salvare i file raw, garantendo ordine nel filesystem.
- **generate_commands**: metodo astratto che converte il dizionario di configurazione in una lista di stringhe di comando (es. applicando i flag corretti per Nmap o Masscan).

3.2.2 Classe **BaseParser**

Simmetricamente, questa classe gestisce la normalizzazione dell'output.

```
class BaseParser(ABC):
    @staticmethod
    @abstractmethod
    def can_handle(file_path: Path) -> bool:
        """
        Returns True if this parser recognizes the file.
        Checks extensions or specific file signatures.
        """
        pass
    @abstractmethod
    def parse(self, file_path: Path) -> dict:
        """
        Parses the file and returns a standardized dictionary.
        """
        pass
```

- **can_handle(file_path)**: metodo statico che identifica se il parser può leggere un determinato file (controllo estensione/header).
- **parse(file_path)**: estrae i dati dal formato proprietario (XML, JSON, Grepable) e restituisce un dizionario Python standardizzato.

3.2.3 Standardizzazione del Modello Dati (Data Transfer Objects)

Un elemento architetturale cruciale per la solidità e l'estendibilità del framework è l'adozione del pattern **Data Transfer Object** (DTO) per la rappresentazione interna delle informazioni. Invece di demandare il passaggio dei risultati tra i moduli (parser, motore di aggregazione e generatore di report) a dizionari non strutturati, il sistema si affida a classi fortemente tipizzate.

Questa implementazione elimina l'ambiguità dei dati scambiati, garantendo un flusso interno esplicito e **type-safe**. Inoltre, l'utilizzo dei DTO stabilisce un «contratto» strutturale rigoroso che semplifica drasticamente lo sviluppo di futuri parser: per integrare un nuovo strumento di scansione, lo sviluppatore deve unicamente mappare l'output grezzo (**raw data**) all'interno delle classi predefinite, garantendone la compatibilità immediata con le fasi di **merging** e **reporting**.

Il modello dati interno si articola sulle seguenti strutture principali:

- **ParsedDataDTO**: Contenitore di livello superiore per l'output di un singolo parser (include nome del tool, comando eseguito e payload dei dati).
- **HostDTO**: Entità logica che aggrega le informazioni relative a un singolo target (IP, hostname, OS, porte rilevate, CPE).
- **PortDTO**: Struttura granulare per i dettagli del servizio esposto (porta, stato, nome servizio, banner catturato, metadati sulla sorgente e TTL).
- **OsDTO**: Metadati specifici per l'identificazione e l'affidabilità del sistema operativo rilevato.
- **FinalReportDTO**: Struttura dati unificata risultante dalla fusione dei risultati, pronta per l'esportazione finale.

Di seguito viene riportato un esempio pratico della costruzione di un record standardizzato, che evidenzia la granularità e l'ordine gerarchico delle informazioni all'interno del framework:

```
from module.dtos.HostDTO import HostDTO
from module.dtos.ParsedDataDTO import ParsedDataDTO
from module.dtos.PortDTO import PortDTO

# Inizializzazione dell'entità Host
host = HostDTO(ip="192.168.1.10")

# Mappatura strutturata di un servizio rilevato
host.ports["22/tcp"] = PortDTO(
    port="22/tcp",
    state="open",
    service="ssh",
    banner="OpenSSH",
    source="nmap",
    ttl=None,
    reason=None,
    cpes=[],
    command="nmap -sV 192.168.1.10"
)

# Inserimento nel payload di esportazione del parser
result = ParsedDataDTO(
    tool_name="nmap",
    command="nmap -sV 192.168.1.10",
    data={"192.168.1.10": host}
)
```

3.3 Pipeline di Esecuzione

3.3.1 Modulo di Orchestrazione (Launcher)

Il cuore operativo, gestito da `main.py` e `module/launcher.py`, orchestra il ciclo di vita delle scansioni. Gli step fondamentali sono:

1. **Setup Ambiente:** creazione della directory di sessione in `output/raw/<timestamp_target>/.</code>`
2. **Generazione Task:** richiama i `CommandGenerator` attivi per creare la lista dei comandi (in base alla configurazione impostata nel `config.yml`).
3. **Esecuzione Concorrente:** i comandi vengono lanciati utilizzando un pool di thread configurabile. L'orchestratore attende la terminazione di tutti i processi.
4. **Incapsulamento (Wrapping):** poiché alcuni tool (es. Masscan, Smap) producono JSON senza metadati sufficienti sul comando generatore, il launcher «incapsula» i file raw aggiungendo il comando che lo ha prodotto (`{«command»:«...», «data»: [...]}`).
5. **Output e Aggregazione:** il modulo restituisce la lista dei file appena generati. È in questa fase di transizione che il sistema inietta nella lista di lavoro eventuali file di scan pregressi (definiti nel config), unificando di fatto i flussi di dati «live» e «storici» in un unico input per il parser.

3.3.2 Modulo di Parsing

Questo componente funge da ponte tra i dati grezzi eterogenei e la logica di business del tool. Riceve in input una lista di percorsi file (che include sia i risultati appena generati dal Launcher, sia eventuali file storici importati da configurazione nel main) ed esegue una trasformazione sequenziale.

Gli step operativi sono:

1. **Iterazione e Dispatching**: il modulo scorre la lista dei file in ingresso. Per ciascuno di essi, analizza i metadati (inclusi quelli aggiunti durante il *wrapping*) o gli header interni per identificare univocamente il formato e istanziare la classe `BaseParser` corretta (pattern *Strategy*).
2. **Normalizzazione**: il parser selezionato (es. `NmapParser` o `MasscanParser`) elabora il contenuto raw e lo converte in un dizionario Python con struttura standardizzata, astruendo le differenze sintattiche tra i vari output proprietari (es. XML vs JSON).
3. **Collezione**: i dizionari normalizzati vengono accumulati in una lista unica (Buffer) che viene infine restituita come input diretto per il successivo Modulo di Fusione.

3.3.3 Modulo di Fusione (Merger)

Implementato in `utils/merge.py`, questo componente rappresenta il collettore finale dell'architettura. Ricevendo in input una lista di dizionari già normalizzati (standardizzati dai Parser), il modulo applica una logica di aggregazione agnostica rispetto allo strumento che ha generato il dato originale.

Le caratteristiche distintive di questo modulo sono:

1. **Astrazione ed Estensibilità**: poiché lavora su oggetti standardizzati e non sui formati raw proprietari, la logica di merge rimane valida ed efficace indipendentemente dalla combinazione di tool lanciati o dall'aggiunta di nuovi plugin in futuro. Questo disaccoppiamento è il pilastro che garantisce la scalabilità del progetto.
2. **Consolidamento «Best Record»**: il sistema non si limita a unire le liste, ma risolve attivamente le sovrapposizioni. Gestisce i duplicati provenienti sia da tool diversi (es. `Masscan` vs `Nmap`) sia da task diversi dello stesso tool (es. *Discovery* vs *Deep Scan*). L'algoritmo confronta le entry per la stessa porta/protocollo e conserva sempre quella più «ricca» di informazioni (es. scartando un semplice stato «open» in favore di un banner completo con versione del servizio).
3. **Schema Unificato**: l'output finale è un oggetto JSON strutturato secondo uno schema rigido e coerente (`result_<ip>.json`). Questo garantisce una visione complessiva chiara dell'infrastruttura target e fornisce un formato deterministico, essenziale per l'ingestione dei dati in dashboard o pipeline di automazione successive.

3.4 Interfaccia Utente (CLI) e Configurazione

L'architettura del tool separa nettamente la logica di esecuzione (codice) dai parametri operativi (configurazione), offrendo un'interfaccia a riga di comando essenziale ma potente.

3.4.1 Esecuzione da Terminale

L'interazione avviene tramite CLI, progettata per richiedere un input minimo da parte dell'operatore. Il comando standard per avviare una sessione di analisi è:

```
sudo python3 main.py <target>
```

L'utilizzo dei privilegi di amministrazione (**sudo**) non è opzionale ma **strettamente necessario** per ragioni architetturali legate ai tool sottostanti:

- **Masscan:** per raggiungere le sue elevate velocità, necessita di bypassare lo stack TCP/IP standard del sistema operativo interagendo direttamente con il driver di rete per l'iniezione di pacchetti raw.
- **Nmap:** le tecniche di scansione più efficaci e furtive (come la *SYN Scan -sS* utilizzata nel profilo standard) e il rilevamento del sistema operativo richiedono l'apertura di socket raw, permesso negato agli utenti non privilegiati.

3.4.2 File di Configurazione (**config.yml**)

Mentre la CLI gestisce l'input del target, il **come** viene eseguita la scansione è interamente governato dal file **config.yml**. Questo file agisce come «pannello di controllo» centralizzato, strutturato in blocchi logici che permettono di:

- **Definire i Profili:** creare set di comandi predefiniti (es. «Discovery Veloce», «Full Audit») attivabili o disattivabili tramite semplici flag booleani (**enabled: true/false**).
- **Gestire i Percorsi:** specificare i path assoluti degli eseguibili o le directory di output, rendendo il tool portabile su diverse distribuzioni senza modificare il codice.
- **Calibrare le Performance:** impostare il numero di thread paralleli e i rate limit (pacchetti/secondo) per adattare l'aggressività del tool alla larghezza di banda disponibile.

4 – Configurazione e Caso d'Uso

4.1 Definizione dello Scenario

Per validare la flessibilità del sistema, è stato definito uno scenario di scansione ibrida. L'obiettivo è orchestrare diversi strumenti in parallelo per ottenere un profilo completo del target, minimizzando i tempi di esecuzione e massimizzando la ricerca delle informazioni sui target.

Il comportamento del framework non è hard-coded nel sorgente, ma è interamente definito nel file `config.yml`. Questo approccio permette di modificare la strategia di scansione (es. passare da una modalità «stealth» a una «aggressive») semplicemente modificando poche righe di testo. Inoltre permette ad un utente di poter aggiungere, rimuovere o modificare i parametri delle varie configurazioni dei tools che poi verranno lanciati. Il tutto senza dover andare a scrivere una singola linea di codice.

Di seguito si analizzano le sezioni chiave della configurazione utilizzata per il test.

4.2 Analisi della Configurazione

L'approccio alla configurazione è modulare: ogni strumento ha la propria sezione dedicata, ma condivide una struttura logica comune basata su «thread» e «modalità».

4.2.1 1. Gestione delle Risorse e Parallelismo

La prima sezione del file definisce le impostazioni globali che regolano il flusso di esecuzione e la gestione dell'output.

```
mode:
  enable_scan: true
  n_threads: 3 # Parallel execution of 3 separate tasks
  base_output_raw_path: "output/raw"
```

Questa configurazione controlla tre aspetti critici:

- `enable_scan`: agisce come interruttore principale per bypassare la scansione attiva, consentendo la rielaborazione di dati storici o importati senza interagire nuovamente con la rete target.
- `n_threads`: imposta il modulo **Launcher** per avviare tre processi simultanei. Questo parallelismo ottimizza i tempi: mentre un thread esegue un task lento (es. Nmap deep scan), altri thread possono completare task veloci (es. Discovery) senza bloccare la pipeline.
- `base_output_raw_path`: definisce la directory radice all'interno della quale il sistema creerà automaticamente le sottocartelle (nominate) per ogni sessione, garantendo un'organizzazione ordinata e gerarchica dei file grezzi (.xml, .json) prodotti dai vari tool.

4.2.2 2. Definizione dei Task (Esempio Nmap)

La sezione più potente è quella relativa ai singoli moduli. Ogni voce sotto la chiave `modes` rappresenta un task atomico che il sistema può assegnare a un thread disponibile.

```
launcher_nmap:
  enabled: true
  top_port_range: 100
  modes:
    # Task 1: Discovery veloce (Ping Sweep)
    - name: scan-only
      enable: true
      flags: "-sn"
      outputpath: "nmap_hostonly.xml"
      custom_option: ""

    # Task 2: Analisi approfondita (Top 100 TCP)
    - name: toptcpsport
      enable: true
      flags: "-sS --top-ports"
      outputpath: "nmap_top_tcp.xml"
      custom_option: ""
```

In questo snippet si nota la granularità del controllo:

- Il task **scan-only** utilizza il flag `-sn` per individuare solo gli host vivi (Ping Sweep), producendo un output rapido.
- Il task **toptcpsport** esegue una scansione SYN (`-sS`) sulle 100 porte più comuni (configurazione ricavata da `top_port_range`).

L'utente può abilitare o disabilitare questi task semplicemente cambiando `enable: true` in `false`, creando profili di scansione personalizzati (es. «Solo Discovery» o «Deep Scan») senza dover modificare il codice Python sottostante. Inoltre è possibile aggiungere parametri custom ad ogni singolo task in modo da avere una capillarità di configurazione massima.

4.2.3 3. Parametri Specifici (Esempio Masscan)

Alcuni strumenti richiedono configurazioni particolari che non sono standard per tutti i tool (es. rate limiting avanzato). Il file di config permette di passare questi parametri direttamente al `CommandGenerator` del modulo specifico.

```
launcher_masscan:
  enabled: true
  rate: 1000 # Pacchetti al secondo
  modes:
    - name: banner-grabber
      enable: true
      flags: "--top-ports --banners"
      outputpath: "masscan_banners.json"
```

Qui viene definito il `rate` (velocità di trasmissione pacchetti), un parametro critico per Masscan. Il sistema leggerà questo valore e costruirà il comando shell includendo automaticamente l'opzione `--rate 1000`, garantendo che l'esecuzione rispetti i limiti di banda della rete.

4.3 Struttura dell'Output e Analisi dei Risultati

L'output finale del tool non è una semplice lista di porte, ma un report strutturato progettato per l'auditing e l'analisi forense. Il file JSON generato (`final_report.json`) si articola in due sezioni macroscopiche:

1. **Metadati Globali:** riporta il numero totale di host coinvolti e, cruciale per la riproducibilità, l'elenco esatto dei comandi raw (Nmap, Masscan, Smap) eseguiti dal sistema durante la sessione.
2. **Dettaglio Host:** un dizionario indicizzato per indirizzo IP che aggrega tutte le informazioni raccolte.

A differenza di molti tool che sovrascrivono i dati, il sistema adotta una logica di **Accumulo delle Evidenze**: per ogni porta rilevata, viene mantenuta una lista di tutti i risultati forniti dai diversi sensori.

4.3.1 Esempio di Report (Mock)

Di seguito viene riportato un estratto anonimizzato di un report reale.

```
{
  "total_hosts": 1,
  "commands": [
    "nmap -sS --top-ports 100 ... 10.10.50.0/24",
    "sudo masscan --top-ports 100 --banners ... 10.10.50.0/24",
    "smap -p1-65535 ... 10.10.50.0/24"
  ],
  "hosts": {
    "10.10.50.15": {
      "ip": "10.10.50.15",
      "hostnames": {
        "web-prod-01.internal.corp": ["nmap", "smap"]
      },
      "commands": [
        "nmap -sS --top-ports 100 ... 10.10.50.0/24",
        "sudo masscan --top-ports 100 --banners ... 10.10.50.0/24",
        "smap -p1-65535 ... 10.10.50.0/24"
      ],
      "ports": {
        "80/tcp": [
          {
            "source": "nmap",
            "state": "open",
            "service": "http",
            "banner": null
          },
          {
            "source": "masscan",
            "state": "open",
            "service": "http",
            "banner": "HTTP/1.1 200 OK\r\nServer: openresty\r\nDate: Thu, 22 Jan 2026...",
            "ttl": 56
          }
        ],
        "1000/tcp": [
          {
            "source": "smap",
            "state": "unknown",
            "service": "ock?",
            "reason": "shodan-api",
            "banner": { "product": null }
          }
        ]
      }
    }
  ]
}
```

4.3.2 Analisi dei Dati

Dall'analisi del JSON sopra riportato emergono i punti di forza dell'architettura:

- **Correlazione Fonti:** la porta 80/tcp contiene due oggetti. Nmap conferma lo stato, ma è Masscan ad aver fornito il banner **Server: openresty**, offrendo un dettaglio prezioso senza richiedere una scansione -sV completa di Nmap.
- **Intelligence Passiva:** la porta 1000/tcp è presente solo grazie a Smap (**source: "smap"**). Nessun pacchetto inviato da Nmap o Masscan l'ha rilevata (probabilmente filtrata dal firewall attuale), ma la sua presenza nel report segnala all'analista una potenziale superficie di attacco storica da investigare.
- **Tracciabilità:** il campo **commands** interno all'host permette di sapere esattamente quali tool hanno interagito con quello specifico IP, garantendo un audit trail completo delle operazioni svolte.

5 – Conclusioni

5.1 Raggiungimento degli obiettivi

Il progetto **A.R.E.S.** ha soddisfatto i requisiti iniziali, fornendo un framework funzionante per l'automazione della ricognizione. L'architettura modulare basata su **CommandGenerator** e **BaseParser** si è rivelata vincente, permettendo l'integrazione fluida di tool eterogenei. La CLI e il sistema di configurazione rendono il tool immediatamente utilizzabile in contesti operativi.

5.2 Limiti attuali e Sviluppi futuri

Attualmente il sistema è vincolato all'esecuzione su Linux nativo per le dipendenze di basso livello di Masscan. Inoltre, la logica di merge, sebbene funzionale, potrebbe essere raffinata con algoritmi di weighting più sofisticati per gestire conflitti complessi.

Gli sviluppi futuri prevedono un'evoluzione della piattaforma verso una maggiore usabilità e portabilità:

- **Sviluppo di una Web GUI Integrata:** realizzazione di un'interfaccia grafica unificata che funga sia da pannello di controllo per la creazione visuale dei file di configurazione (wizard), sia da dashboard analitica per la visualizzazione interattiva del report JSON (grafici, tabelle filtrabili e mappe di rete).
- **Estensione dei Moduli di Analisi:** integrazione di strumenti per il **Vulnerability Assessment** automatizzato (es. Nuclei) e aggiunta di nuove sorgenti di intelligence passiva, trasformando il tool da semplice scanner a piattaforma di **Continuous Security Monitoring**.
- **Modalità «Offline Ingestion»:** implementazione di un modulo per il parsing asincrono («Wrapping»), capace di ingerire e normalizzare file di output prodotti da scansioni eseguite esternamente al framework, permettendo di centralizzare dati storici o provenienti da terze parti.
- **Containerizzazione e Supporto Cross-Platform:** creazione di immagini Docker ottimizzate per garantire l'esecuzione agnostica rispetto al sistema operativo ospitante. Questo approccio mirerà a risolvere le dipendenze di basso livello (es. accesso ai raw socket per Masscan), estendendo la compatibilità anche ad ambienti Windows/macOS tramite virtualizzazione leggera.

In conclusione, **A.R.E.S.** rappresenta una solida base per l'ottimizzazione delle attività di Red Teaming, liberando gli operatori dal carico delle operazioni ripetitive di raccolta dati, permettendo loro di focalizzarsi solamente sull'analisi delle criticità. Inoltre, la generazione di output strutturati e normalizzati trasforma il framework in un elemento abilitante per l'intera pipeline di Network Pentesting, ponendo le fondamenta per l'automazione delle fasi successive di **Vulnerability Assessment** ed **Exploitation**.